Bocconi

# COMPUTER SCIENCE

code 30424 a.y. 2024-2025

## Lesson 24

Graphs and Minimum Spanning Tree

Università Bocconi
MILANO

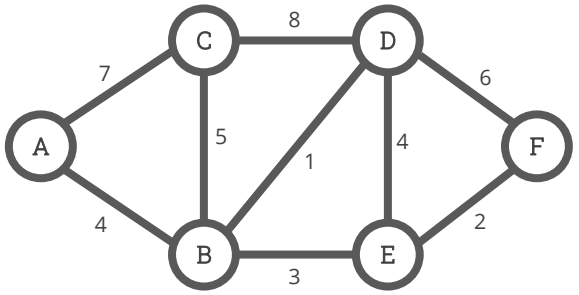
# Objectives of the lesson

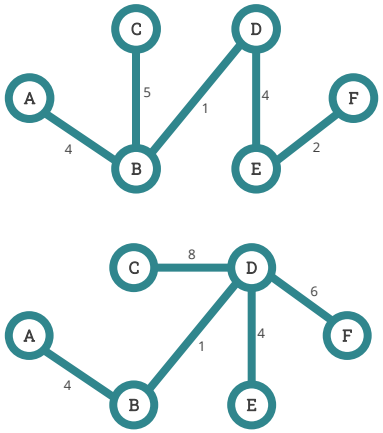
- Understand what are Spanning Trees
- Learn what are Greedy algorithms
- Understand how Minimum Spanning Tree algorithms work

# Spanning trees

A spanning tree is a **sub-graph** of an undirected connected graph, connecting all the vertices using the **lowest number** of edges. Therefore, a spanning tree is the path connecting all the vertices with the minimum length. Moreover:

- ✓ if a vertex is not considered, then it is not a spanning tree
- ✓ a graph may have more than one spanning tree
- ✓ a spanning tree does not have cycles



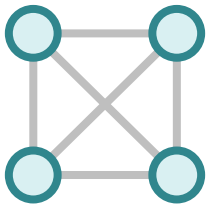



# Graphs and spanning trees

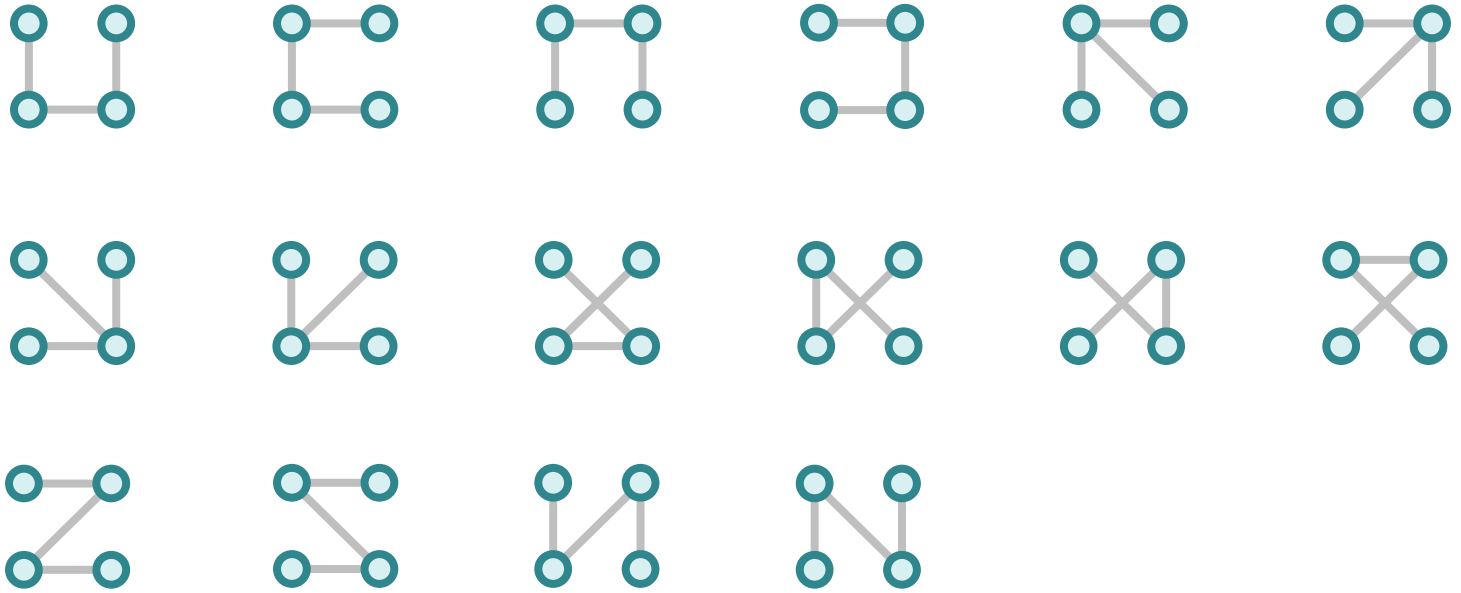
- The number of spanning trees in a complete graph with **N** vertices is $\mathbf{N^{N-2}}$
- For a connected graph having **N vertices**, the number of **edges** in the spanning tree for that graph will be $\mathbf{N - 1}$

Graph

*(4 vertices, 6 edges)*



All possible spanning trees ( $4^{4-2} = 16$ ), each with 3 edges



# Minimum spanning tree

- A common operation in weighted undirected connected graphs is to calculate the **minimum spanning tree** (**MST**), which is a spanning tree with the minimum weight among all the possible spanning trees of the graph
- The minimum weight of the spanning tree is given by the **lowest sum of the weights** of the edges

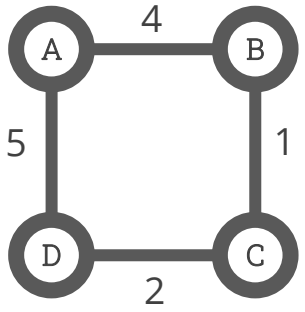
# Minimum spanning tree properties

A minimum spanning tree has the following properties:

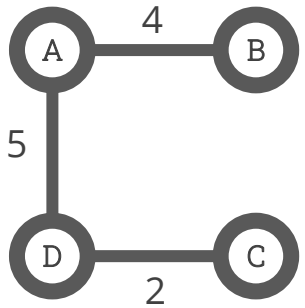
- ✓ it connects **all the vertices** in the graph
- ✓ it is **acyclic**, meaning it contains no cycles (this property ensures that it remains a tree and not a graph with cycles)
- ✓ an MST with N vertices will have exactly **N-1 edges**
- ✓ an MST is optimal for minimizing the total edge weight, but it may **not necessarily** be **unique**

# Minimum spanning tree example

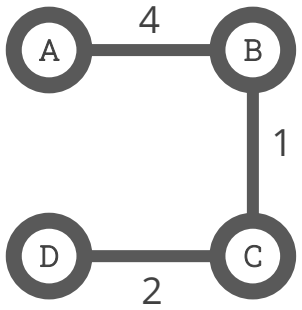
Consider the following undirected connected graph with weights:




The possible spanning trees from the above graph are:

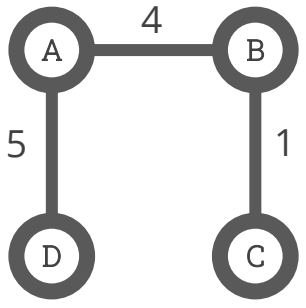



**Sum = 11**

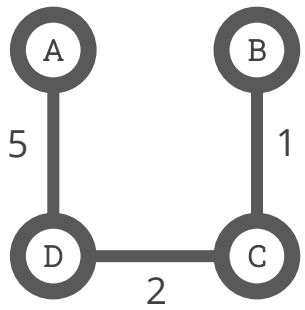



**Sum = 7**

Minimum Spanning Tree




**Sum = 10**




**Sum = 8**

# Greedy search and greedy algorithms

- **Greedy search** is the approach that always picks what looks best at the moment
- There are various algorithms to solve MST and other similar problems, and most of them are "**greedy algorithms**"
- Greedy algorithms find an optimal solution to the problem building it up piece by piece, by choosing the **best choice at each step** (the "greedy" choice) without reconsidering the previous steps once chosen

# Greedy algorithm example

We have to make a change of an amount using the **smallest number of coins**. The change to make is **18 cents**. The available coins are: 1¢, 2¢ and 5¢. There is no limit to the number of each coin we can use.

A greedy algorithm steps will be the following:

- ✓ create an empty “solution set” to contain answers (coins)
- ✓ at each step, a coin is added to the “solution set”. The algorithm always selects the coin with the largest value hoping to reach the destination faster (this is the greedy choice)
- ✓ if the sum does not exceed the total to be reached, the coin is added to the solutions, otherwise the coin is rejected and never considered again
- ✓ the choice is repeated until a solution is reached

# Greedy algorithm example (solution)

- We create an empty "solution set" (i.e. **solution = [ ]** ) and a "coins set" (i.e. **coins = (5, 2, 1)** )
- We are supposed to find a sum = 18, starting with sum = 0
- We select the coin with the largest value (i.e. 5) until the sum > 18
- In the first iteration, **solution = [5]** and **sum = 5**
- In the second iteration, **solution = [5, 5]** and **sum = 10**
- In the third iteration, **solution = [5, 5, 5]** and **sum = 15**
- In the fourth iteration, **solution = [5, 5, 5, 2]** and **sum = 17** (we cannot select 5 because otherwise sum = 20, which is greater than 18. So, we select the second largest coin which is 2)
- Similarly, in the fifth iteration, we select 1: now **sum = 18** and **solution = [5, 5, 5, 2, 1]**

# Greedy algorithm example (2)

We have to make a change of an amount using the **smallest number of coins**. The change to make is always **18 cents**, but this time the available coins are: 2¢, 5¢ and 10¢.

Our greedy algorithm steps will be the following:

- We create: solution = [ ] , coins (10, 5, 2) and sum = 0
- We select the coin with the largest value until the sum > 18, so in the first iteration, **solution = [10]** and **sum = 10**
- In the second iteration, **solution = [10, 5]** and **sum = 15**
- In the third iteration, **solution = [10, 5, 2]** and **sum = 17**
- In the fourth iteration, **the algorithm fails**: we cannot select 2 because if we do so, sum = 19 which is greater than 18... So the algorithm gets stuck at 1 cent remaining

# Greedy algorithm exercise

You are asked to use a **greedy approach** to make change for a given amount using the smallest possible number of coins, considering that:

- ✓ the amount to make change for is **5.89** euros
- ✓ the available coins are: 1¢, 2¢, 5¢, 10¢, 20¢ and 50¢, 1€ and 2€. There is no limit to the number of each type of coin you can use

You can write on paper the steps taken and the coins used to solve the problem.

# Greedy algorithm exercise (step-by-step solution)

1) We use 2€ coins (200¢): 589 // 200 → 2 (remaining: 589 - 400 = 189¢)
2) We use 1€ coins (100¢): 189 // 100 → 1 (remaining: 189 - 100 = 89¢)
3) We use 50¢ coins: 89 // 50 → 1 (remaining: 89 - 50 = 39¢)
4) We use 20¢ coins: 39 // 20 → 1 (remaining: 39 - 20 = 19¢)
5) We use 10¢ coins: 19 // 10 → 1 (remaining: 19 - 10 = 9¢)
6) We use 5¢ coins: 9 // 5 → 1 (remaining : 9 - 5 = 4¢)
7) We use 2¢ coins: 4 // 2 → 2 (remaining: 4 - 4 = 0¢)

**Final Answer (9 coins):** $2 \times 2€ + 1 \times 1€ + 1 \times 50¢ + 1 \times 20¢ + 1 \times 10¢ + 1 \times 5¢ + 2 \times 2¢$

# Greedy algorithm exercise

In the **make_change.py** file you can find two incomplete function to calculate the change. You are asked to complete them considering that the first function:

- has the amount of the change as a mandatory parameter
- creates the coins_set tuple with all the available coins
- using a greedy approach, must calculate and store in the list change the coins used to make the change
- return the list change

Possible list change could be [2, 1, 0.20, 0.20, 0.05, 0.01] or [1, 0.20, 0.10, 0.05, 0.02]

The second function must calculate the change as well, but must return the dictionary change with coin denominations as keys and the count of the coin as values.

# MST algorithms

- Given an undirected connected graph with weights, there are various algorithms to find the minimum spanning tree
- The most used ones are **Kruskal's** and **Prim's** algorithms: they both are greedy algorithms and they both find an optimal solution as long as the graph meets the following assumptions:
  - ✓ the graph must be **undirected**
  - ✓ the graph must be **connected**: if the graph is disconnected, these algorithms return a Minimum Spanning Forest (a set of MSTs for each connected component) and not a single MST
  - ✓ edge weights must be **non-negative** and **unique** between the two nodes considered (if they're not unique the algorithms may still find a valid MST, but it may not be unique)
  - ✓ **no extra constraints** are allowed, other than edge weights that determine the cost

# Kruskal's MST algorithm

- Given an undirected connected graph with weights, Kruskal's MST algorithm finds the spanning tree which:
  - ✓ contains all the vertices (with no cycles)
  - ✓ has the minimum sum of weights
- Kruskal's algorithm uses a greedy approach, meaning it makes optimal choices at each step to achieve an optimal solution

# Kruskal's MST algorithm

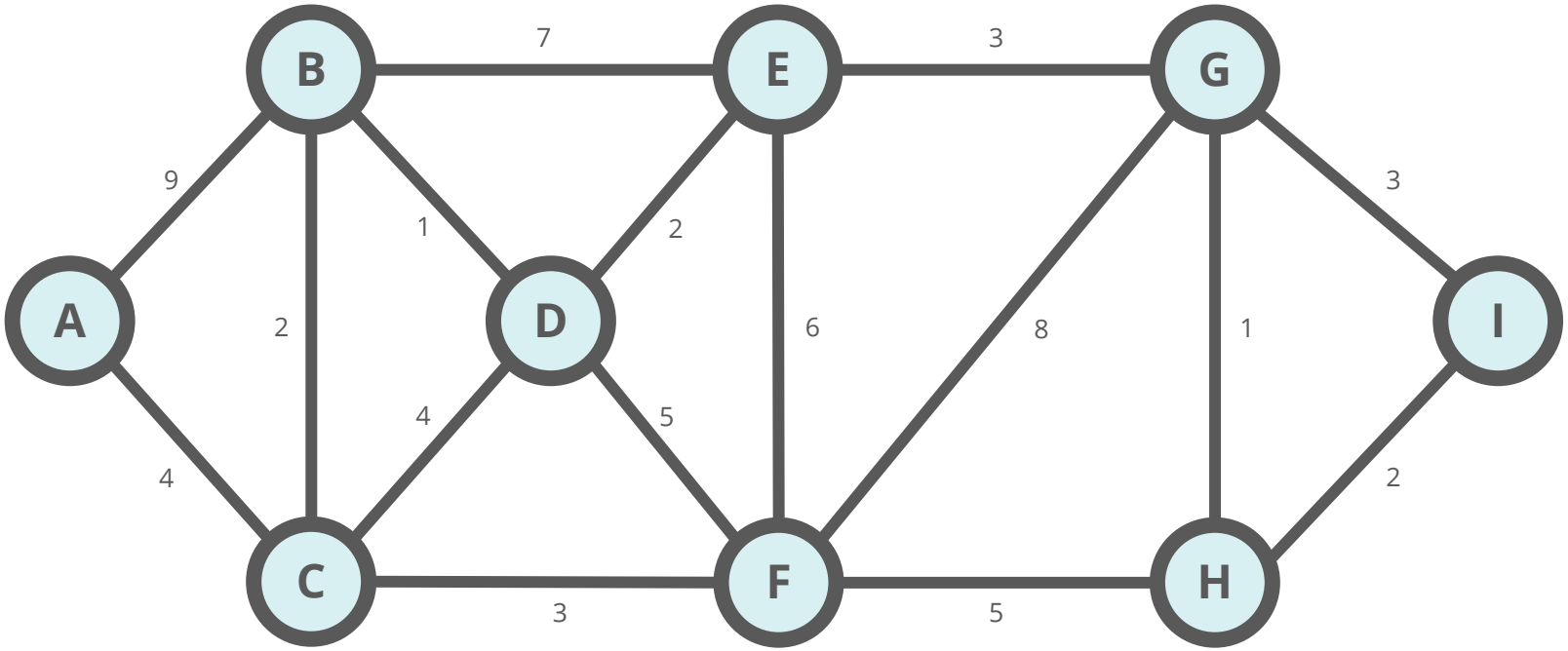
Kruskal's algorithm starts from the edge with the lowest weight and keeps adding edges until it finds the Minimum Spanning Tree.

The algorithm follows these steps:

- ✓ it sorts all the edges of the graph by their weights (in ascending order)
- ✓ it takes the edge with the lowest weight and adds it to the spanning tree
- ✓ it keeps adding edges until it reaches all vertices. If by adding an edge a cycle is created, it rejects the edge and skips to the following one in the sorted list of all the edges
- ✓ it returns the minimum spanning tree

# Kruskal's algorithm example

Consider the following undirected connected graph with weights. We need to find the MST using Kruskal's algorithm

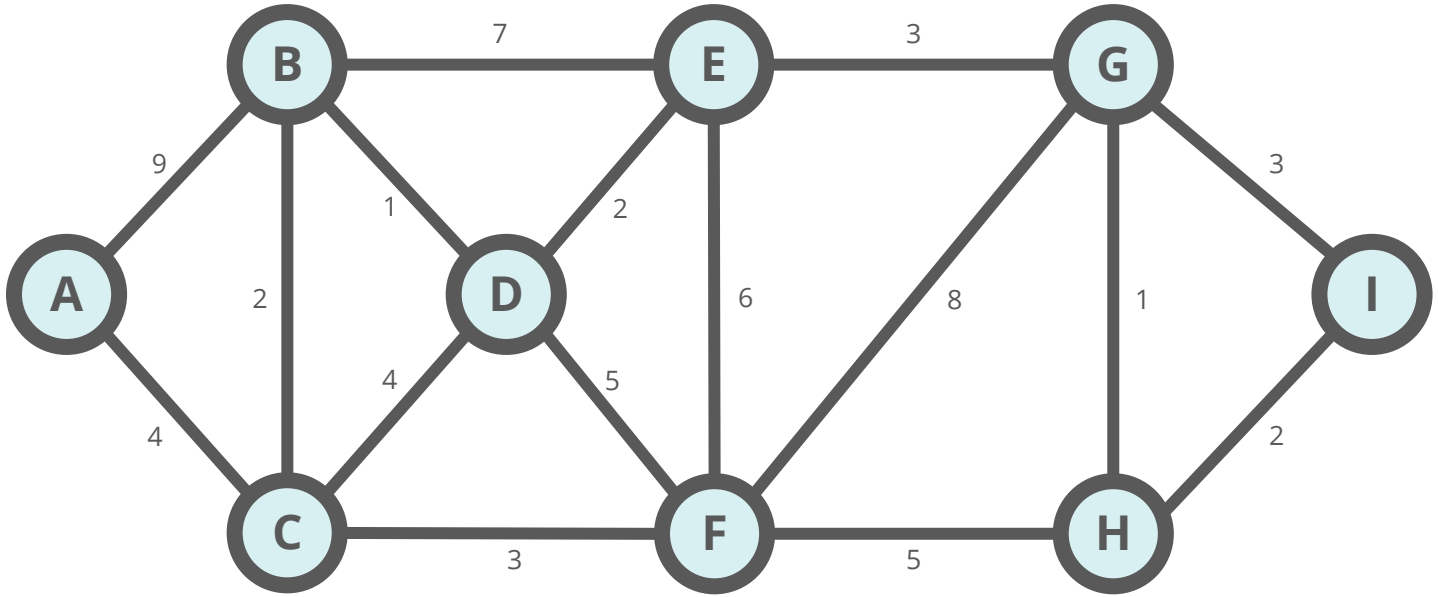


# Kruskal's algorithm example - Step 1

- First we need to sort all the edges by ascending weight. If two or more edges have the same weight, we can arbitrarily choose which one comes first:

Edges:

| | |
|---|---|
| B to D = 1 | A to C = 4 |
| G to H = 1 | C to D = 4 |
| B to C = 2 | D to F = 5 |
| H to I = 2 | F to H = 5 |
| D to E = 2 | E to F = 6 |
| C to F = 3 | B to E = 7 |
| E to G = 3 | F to G = 8 |
| G to I = 3 | A to B = 9 |

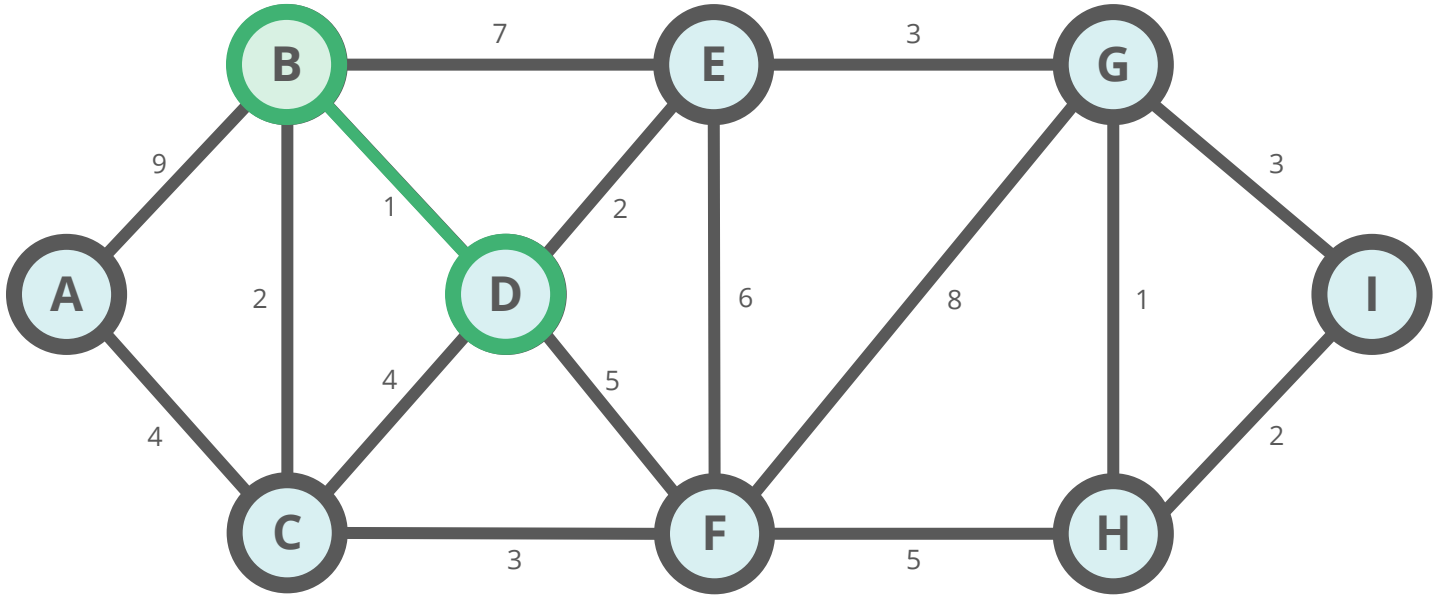


# Kruskal's algorithm example - Step 2

- We start selecting the first edge (the one with the lowest weight) and we add it to the tree

Edges:

| | |
|---|---|
| B to D = 1 | A to C = 4 |
| G to H = 1 | C to D = 4 |
| B to C = 2 | D to F = 5 |
| H to I = 2 | F to H = 5 |
| D to E = 2 | E to F = 6 |
| C to F = 3 | B to E = 7 |
| E to G = 3 | F to G = 8 |
| G to I = 3 | A to B = 9 |

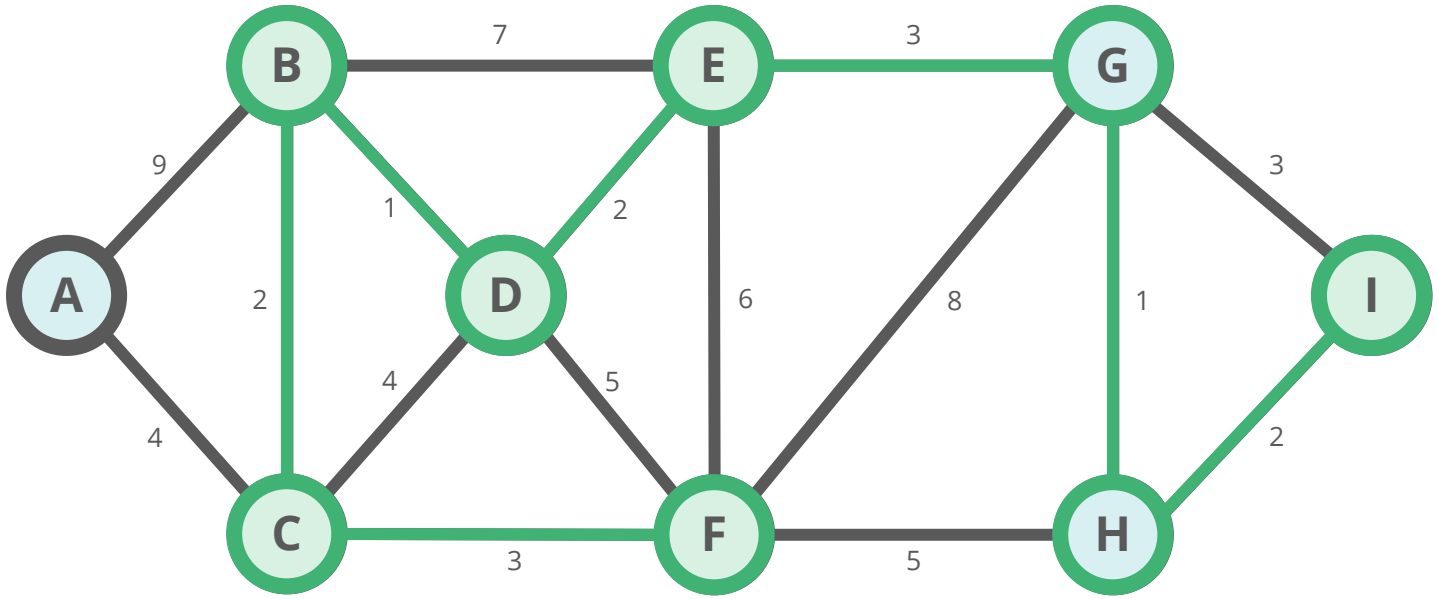


# Kruskal's algorithm example - Step 2

- We keep adding edges (in ascending order) to the tree, checking that no cycle is formed: if no cycle is formed, it is a valid selection, otherwise we skip that edge

Edges:

| | |
|---|---|
| B to D = 1 | A to C = 4 |
| G to H = 1 | C to D = 4 |
| B to C = 2 | D to F = 5 |
| H to I = 2 | F to H = 5 |
| D to E = 2 | E to F = 6 |
| C to F = 3 | B to E = 7 |
| E to G = 3 | F to G = 8 |
| G to I = 3 | A to B = 9 |

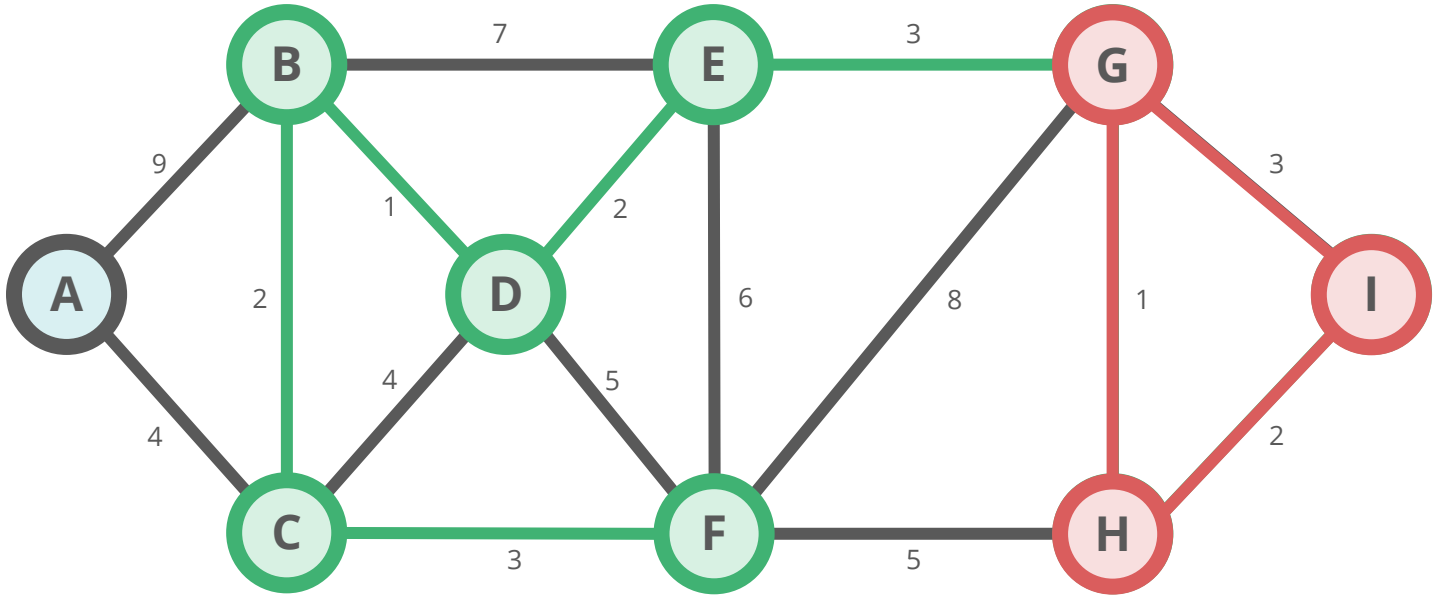


# Kruskal's algorithm example - Step 2

- We keep adding edges (in ascending order) to the tree, checking that no cycle is formed: if not, it is a valid selection, otherwise we skip that edge

Edges:

- B to D = 1
- G to H = 1
- B to C = 2
- H to I = 2
- D to E = 2
- C to F = 3
- E to G = 3
- ~~G to I = 3~~
- A to C = 4
- C to D = 4
- D to F = 5
- F to H = 5
- E to F = 6
- B to E = 7
- F to G = 8
- A to B = 9

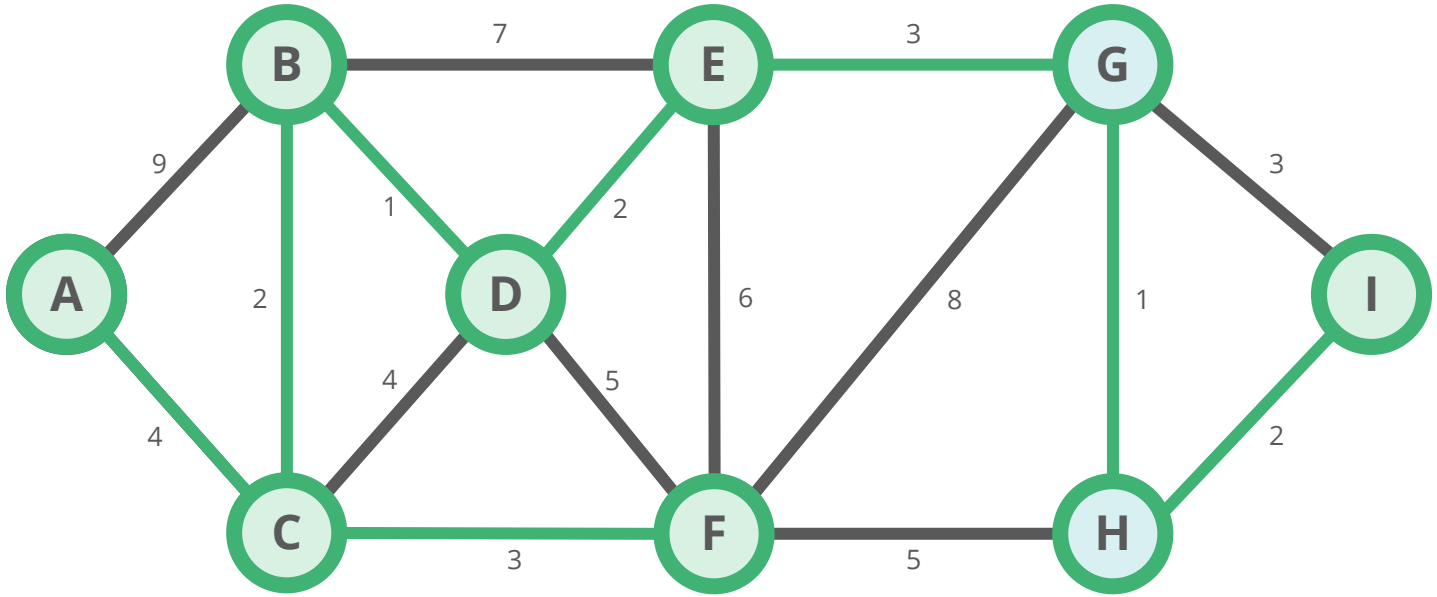


# Kruskal's algorithm example - Step 2

- We keep adding edges (in ascending order) to the tree, checking that no cycle is formed: if not, it is a valid selection, otherwise we skip that edge

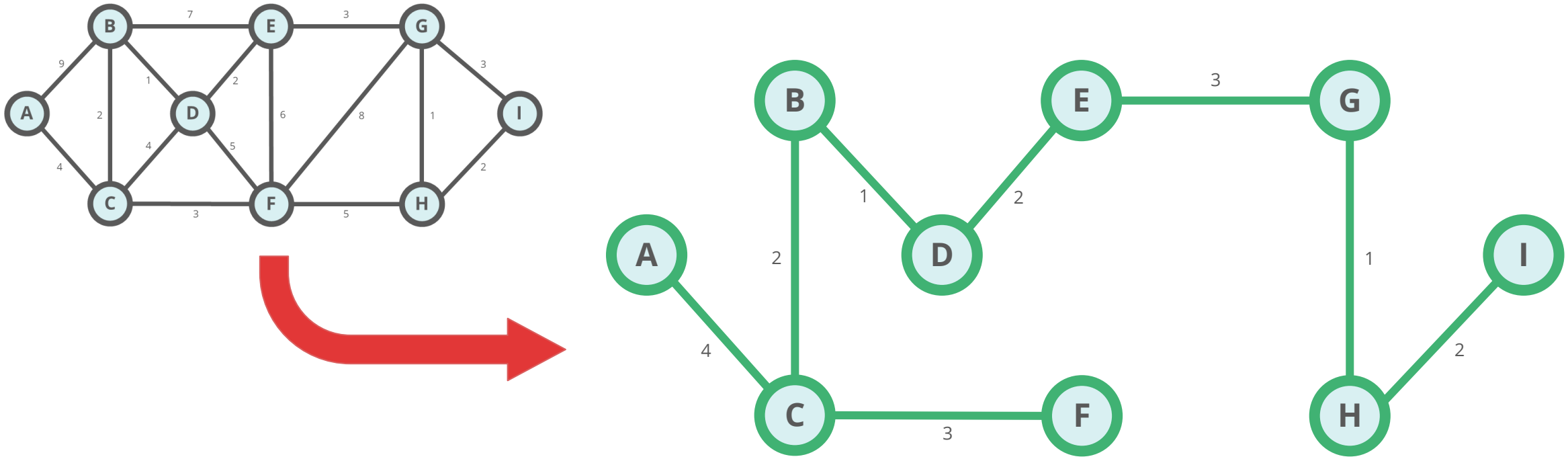
Edges:

| | |
|---|---|
| B to D = 1 | A to C = 4 |
| G to H = 1 | ~~C to D = 4~~ |
| B to C = 2 | ~~D to F = 5~~ |
| H to I = 2 | ~~F to H = 5~~ |
| D to E = 2 | ~~E to F = 6~~ |
| C to F = 3 | ~~B to E = 7~~ |
| E to G = 3 | ~~F to G = 8~~ |
| ~~G to I = 3~~ | ~~A to B = 9~~ |



# Kruskal's algorithm example - Step 3

- After considering all edges, we have the MST. We had 9 vertices and the MST has 8 edges (N-1). The sum of the weights is 18



# Prim's algorithm

- Given an undirected connected graphs with weights, Prim's MST algorithm finds the spanning tree which:
  - ✓ contains all the vertices (with no cycles)
  - ✓ has the minimum sum of weights
- Like Kruskal's algorithm, also Prim's uses a greedy approach, meaning it makes optimal choices at each step to achieve an optimal solution
- The main difference between Prim's and Kruskal's algorithms is that instead of starting from an edge, Prim's algorithm starts from a vertex and keeps adding lowest-weight edges which aren't already in the tree, until all vertices have been covered

# Prim's algorithm

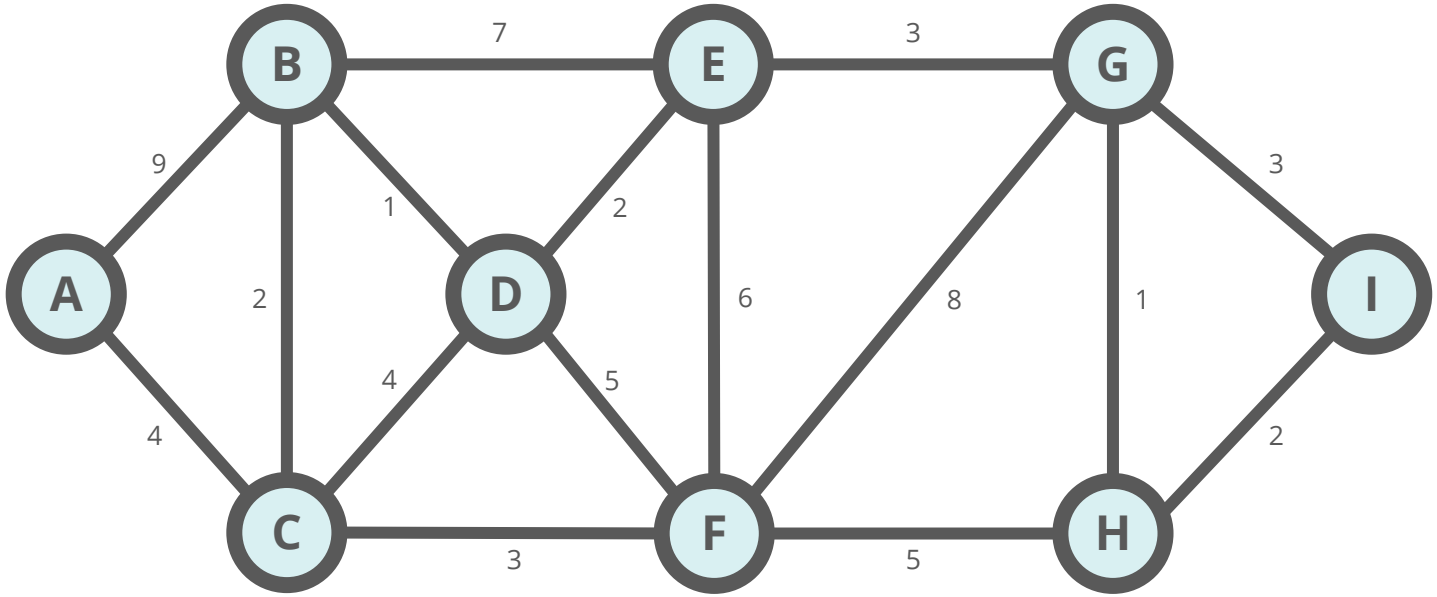
Prim's algorithm starts from a randomly picked vertex and keeps adding edges with the lowest weight until it finds the Minimum Spanning Tree.

The algorithm follows these steps:

- ✓ it initializes the minimum spanning tree with a vertex chosen randomly
- ✓ it finds all the edges that connect that vertex to other vertices not yet in the MST and adds the one with the lowest weight to the MST
- ✓ it keeps repeating the previous step until it reaches all vertices. If by adding an edge to the tree a cycle is created, it rejects that edge and skips to the following one
- ✓ it returns the minimum spanning tree

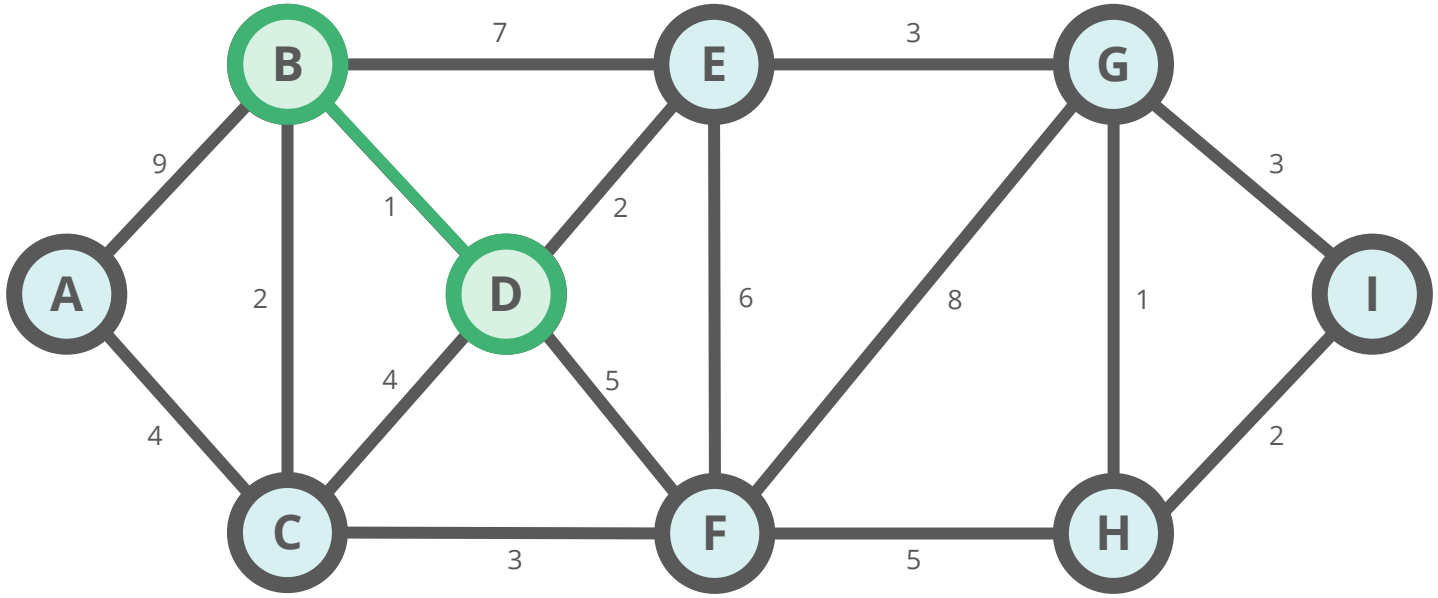
# Prim's algorithm example

Consider the following undirected connected graph with weights. We need to find the MST using Prim's algorithm.



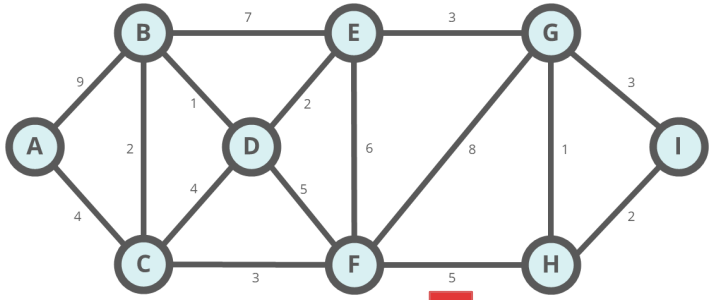
# Prim's algorithm example - Step 1

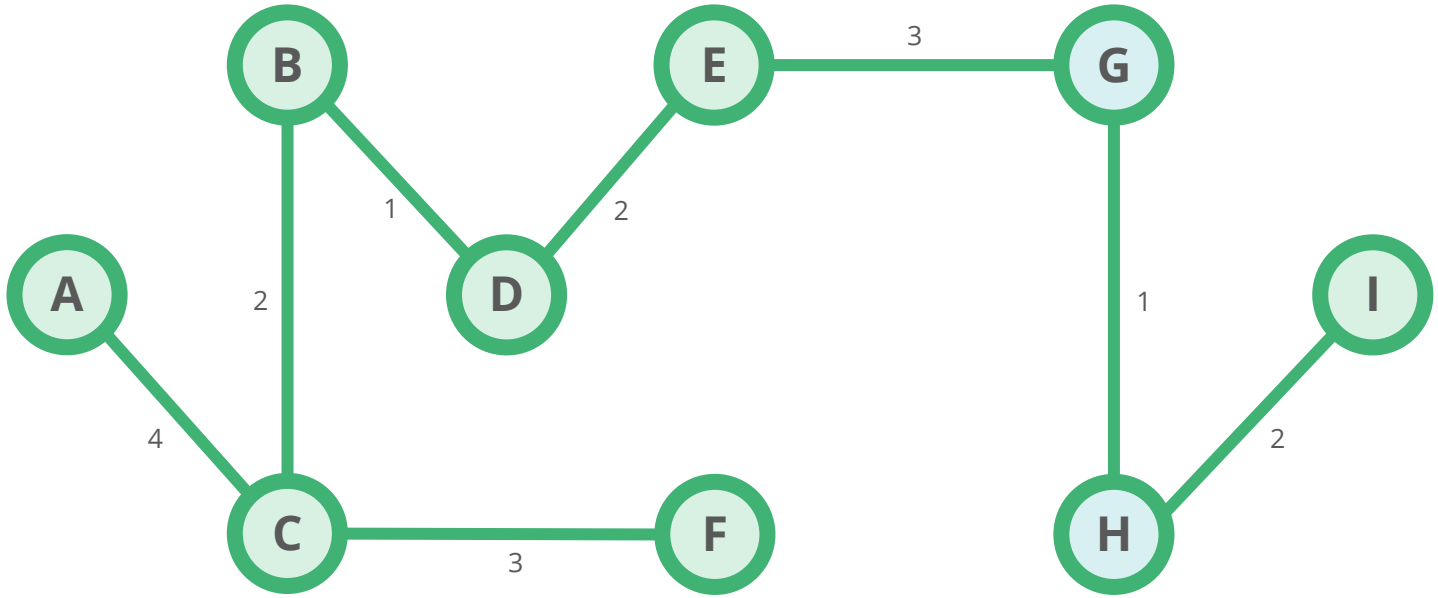
- We choose a vertex to start from: for example D
- From all the vertices connected to D, we add the one with the lowest weight to the MST. In this case it is B (with edge D to B = 1)



# Prim's algorithm example - Step 2

- We continue connecting vertices adding the edge with the lowest weight, until all the vertices are added to the MST
- At each step, if adding an edge to the tree creates a cycle, we reject that edge and skip to the following one
- At the end of the process we have the MST: it has 9 vertices, 8 edges, and the sum of the weights is 18



# Topics to learn with today's lecture

- Spanning Trees
- Greedy algorithms
- Kruskal's and Prim's Minimum Spanning Tree algorithms

# Study materials

This **set of slides** has to be considered the study material for the contents of **Lesson 24**

Any links to external resources have to be considered as supplementary material for further exploration of the topics covered